

ARQUITETURA E ORGANIZAÇÃO DE COMPUTADORES

Laboratório de Circuitos — Projeto Integrador

Parte I — Subsistema de Memória e Cache: Documentação

Campo	Preenchimento
Disciplina	Arquitetura e Organização de Computadores — Semestre 2026.2
Professor	Prof. Dr. Herbert Oliveira Rocha
Integrante 1	Mateus Henrique de Souza Pinheiro
Integrante 2	Fabício Cauã Pereira Silvino
Integrante 3	Darckison Almeida Trajano
Repositório GitHub	https://github.com/MateusHenrique2025/AOC_MateusHenriqueDarckisonAlmeidaFabicio_UFRR_LabCircuitos_2026
Ferramenta	Logisim-Evolution v5.0.0
Data de referência deste rascunho	01/10/2026

1. Introdução e objetivos

Esta parte do projeto constrói o módulo de memória de 64 bytes com decodificação de endereços e a cache de 4 linhas que opera sobre ele, testada sobre um traço de leituras. Os objetivos de aprendizagem tratados são:

- OA1: explicar por que vários dispositivos podem compartilhar um mesmo barramento e qual é o papel do decodificador de endereços nesse arranjo.
- OA2: montar, em Logisim-Evolution, um módulo de memória de 64 bytes com quatro regiões selecionadas por um decodificador $2 \rightarrow 4$.
- OA3: decompor endereços de 6 bits em rótulo, índice e deslocamento, e classificar cada acesso de um traço como acerto, falta compulsória ou falta por conflito.
- OA4: calcular a taxa de acertos e o tempo médio de acesso à memória a partir da planilha do traço.

2. Declaração de uso de ferramentas de IA generativa

Foi utilizada a ferramenta Claude (Anthropic) nas seguintes etapas e finalidades:

- A. Etapas para Construção de cada componente;
- B. Ligações corretas de cada componente;
- C. Comportamento correto de cada circuito;
- D. Hierarquia correta dos arquivos;
- E. Problemas de sincronização;

1. Mapa de Memória

O espaço de endereçamento de 64 bytes (A5..A0) é dividido em quatro regiões de 16 bytes, selecionadas pelos dois bits mais significativos do endereço (A5 e A4) através de um decodificador 2→4 construído a partir de portas AND e NOT (sem uso do componente Decoder pronto da biblioteca). Os quatro bits restantes (A3..A0) endereçam a posição interna de 16 bytes da região já selecionada.

Faixa	Dispositivo	Tamanho	Sinal de seleção
0x00 – 0x0F	ROM de programa	16 bytes	$CS_ROM = A5' \cdot A4'$
0x10 – 0x1F	RAM de dados	16 bytes	$CS_RAM = A5' \cdot A4$
0x20 – 0x2F	Banco de registradores	16 bytes	$RegisteBank = A5 \cdot A4'$
0x30 – 0x3F	Região de E/S	16 bytes	$IOinterface = A5 \cdot A4$

Vale ressaltar que $CS_ROM = A5' \cdot A4'$ não habilita escrita na ROM, os dados podem ser gravados manualmente para um posterior acesso.

As quatro saídas de dados (ROM, RAM, banco de registradores e E/S) convergem para um multiplexador de 4 entradas (Componente 02), cujas linhas de seleção são os próprios bits A5 e A4 — e não os sinais CS —, já que ambos carregam exatamente a mesma informação. Os CS_RAM , RegisterBank e IOinterface são usados para habilitar seletivamente a escrita em cada dispositivo (AND entre o CS correspondente e um sinal global de escrita), evitando que uma operação de escrita afete mais de uma região ao mesmo tempo.

imagem completa:

2. Componente básicos para a construção

2.1. Decodificador 2x4 que permite a escrita

A tabela abaixo comprova que, para cada uma das quatro combinações de A5 e A4, exatamente um sinal CS fica ativo.

Input A5	Input A4	CS_ROM	CS_RAM	CS_RegisteBank	IOinterface
0	0	1	0	0	0
0	1	0	1	0	0

1	0	0	0	1	0
1	1	0	0	0	1

Expressão booleana

Cada saída representa um minitermo (minterm) das variáveis de entrada:

- **CS_ROM** = $\overline{A5} \cdot \overline{A4}$ (Ativa quando as entradas são 00)
- **CS_RAM** = $\overline{A5} \cdot A4$ (Ativa quando as entradas são 01)
- **RegisterBank** = $A5 \cdot \overline{A4}$ (Ativa quando as entradas são 10)
- **IOinterface** = $A5 \cdot A4$ (Ativa quando as entradas são 11)

2.2. MUX 4x1 que seleciona a saída desejada

Seleciona a saída de dados de 8 bits de um dos 4 componentes.

A5	A4	Saída (F)	Descrição
0	0	D_0	Seleciona o CS_ROM
0	1	D_1	Seleciona a CS_RAM
1	0	D_2	Seleciona o CS_RegisteBank
1	1	D_3	Seleciona o IOinterface

Expressão Booleana

A expressão lógica que define a saída F em função das entradas de dados e das variáveis de seleção é a soma dos produtos (mintermo) correspondente a cada linha da tabela verdade:

$$F = \overline{S_1} \cdot \overline{S_0} \cdot D_0 + \overline{S_1} \cdot S_0 \cdot D_1 + S_1 \cdot \overline{S_0} \cdot D_2 + S_1 \cdot S_0 \cdot D_3$$

2.3. Porta Xor

Nesse projeto, a XOR participou do papel de verificar a integridade da palavra armazenada na RAM a partir das saídas do detector de paridade e paridade da leitura.

Entrada A	Entrada B	Saída S	Expressão
0	0	0	—
0	1	1	$\bar{A} \cdot B$
1	0	1	$A \cdot \bar{B}$
1	1	0	—

Regra Prática

- **Entradas iguais** (0 e 0, ou 1 e 1) → Saída $S = 0$.
- **Entradas diferentes** (0 e 1, ou 1 e 0) → Saída $S = 1$ (Problema na integridade dos dados).

2.4 Xor de 8 bits

Verifica a paridade ímpar da saída de um dos componentes da Hierarquia de memória selecionada e da entrada de dados de

Expressão Booleana para um barramento A de **8 bits**

$$(A_7, A_6, A_5, A_4, A_3, A_2, A_1, A_0)$$

A expressão para detectar o número ímpar de bits em um vetor de 8 bits é a soma lógica exclusiva de todos os seus bits:

$$S = A_7 \oplus A_6 \oplus A_5 \oplus A_4 \oplus A_3 \oplus A_2 \oplus A_1 \oplus A_0$$

2.5 Como usar o parte1_memoria.circ

Entradas

Pino	Tamanho	Função
ENDERECO (A5..A0)	6 bits	Seleciona a posição de memória (0x00 a 0x3F). Os bits A5/A4 escolhem a região (ROM, RAM, banco de registradores, E/S); A3..A0 escolhem a posição dentro dela.
DADOS_ENTRADA	8 bits	Valor a ser gravado. Chega em paralelo em todos os dispositivos graváveis (RAM, banco de registradores, E/S), mas só é de fato gravado em quem estiver habilitado no mesmo ciclo.
ESCREVER	1 bit	Habilitação de escrita global. Em 0, nenhum acesso grava nada (só leitura). Em 1, o dispositivo cuja região corresponde ao ENDERECO atual grava DADOS_ENTRADA — os outros três ignoram, mesmo recebendo o mesmo dado no pino. A ROM nunca grava, independente deste pino.
CLOCK	1 bit	Clock único para todos os dispositivos síncronos. Uma escrita só acontece na borda configurada (Trigger) do clock, e só se ESCRIVER estiver em 1 naquele instante.

Saídas

Pino	Tamanho	Comportamento
DADOS_SAIDA	8 bits	Sempre reflete o conteúdo do endereço atual, lido continuamente (não precisa de clock para aparecer — é a saída do MUX, puramente combinacional sobre A5/A4). Depois de uma escrita, a leitura no mesmo endereço já mostra o novo valor a partir do próximo ciclo.

ERRO_PARIDADE 1 bit
E

Só é significativo quando o endereço atual está na faixa da RAM (0x10–0x1F) — nas outras regiões, esse sinal não tem significado. Fica em 0 enquanto a paridade armazenada bate com a paridade recalculada do dado lido. Vai para 1 se o dado na RAM foi corrompido (alterado) sem reescrever a paridade correspondente.

3. Divisão do Endereço de 6 Bits

Cada endereço de 6 bits usado pela cache se divide em três campos, conforme os parâmetros fixos da Tabela 6 (4 linhas, bloco de 2 bytes):

deslocamento = $\log_2(\text{tamanho_do_bloco}) = \log_2(2) = 1 \text{ bit} \rightarrow A_0$

índice = $\log_2(\text{número_de_linhas}) = \log_2(4) = 2 \text{ bits} \rightarrow A_2 A_1$

rótulo = $6 - 2 - 1 = 3 \text{ bits} \rightarrow A_5 A_4 A_3$

Exemplo demonstrativo: o endereço 0x22 corresponde a 100010 em binário. Lendo da direita para a esquerda: deslocamento = 0, índice = 01, rótulo = 100. Esse endereço só pode ocupar a linha 1 da cache, e nela ficaria registrado o rótulo 100.

4. Comparador de Rótulos e Condição de Acerto

O comparador de rótulos foi construído integralmente a partir de portas AND, NOT e OR (Componente 03), sem uso da porta XOR pronta da biblioteca, conforme exigido:

$\text{XOR}(a,b) = (a \text{ AND NOT } b) \text{ OR } (\text{NOT } a \text{ AND } b)$

$\text{iguais} = \text{NOT}((T_2 \text{ XOR } S_2) \text{ OR } (T_1 \text{ XOR } S_1) \text{ OR } (T_0 \text{ XOR } S_0))$

$\text{ACERTO} = \text{iguais AND válido}[\text{índice}]$

Onde $T_2..T_0$ é o rótulo do endereço pedido (A_5, A_4, A_3) e $S_2..S_0$ é o rótulo armazenado na linha indicada pelo índice (A_2, A_1). Os bits de validade das 4 linhas foram implementados com flip-flops D individuais (Componente 01), com entrada de clear comum para permitir invalidar a cache inteira de uma só vez (requisito de ‘bits de validade zeráveis’).

5. Traço Obrigatório de Acessos — Resultados

A tabela abaixo apresenta o resultado esperado, derivado da simulação das regras de mapeamento direto sobre os 16 acessos da Tabela 7, partindo da cache invalidada. Os valores devem ser conferidos contra a execução real do circuito montado em Logisim-Evolution antes da entrega.

Ac.	End. (hex)	End. (bin)	Rótulo	Índice	Desl.	Vál. antes	Rót. antes	Resultado	Tipo falta	Rót. depois
1	0x00	000000	000	00	0	0	traço	FALTA	compulsória	000
2	0x01	000001	000	00	1	1	000	ACERTO	—	000
3	0x02	000010	000	01	0	0	traço	FALTA	compulsória	000
4	0x03	000011	000	01	1	1	000	ACERTO	—	000
5	0x00	000000	000	00	0	1	000	ACERTO	—	000
6	0x20	100000	100	00	0	1	000	FALTA	conflito	100
7	0x21	100001	100	00	1	1	100	ACERTO	—	100
8	0x00	000000	000	00	0	1	100	FALTA	conflito	000
9	0x04	000100	000	10	0	0	traço	FALTA	compulsória	000
10	0x05	000101	000	10	1	1	000	ACERTO	—	000
11	0x06	000110	000	11	0	0	traço	FALTA	compulsória	000
12	0x07	000111	000	11	1	1	000	ACERTO	—	000
13	0x22	100010	100	01	0	1	000	FALTA	conflito	100
14	0x23	100011	100	01	1	1	100	ACERTO	—	100
15	0x02	000010	000	01	0	1	100	FALTA	conflito	000
16	0x03	000011	000	01	1	1	000	ACERTO	—	000

6. Indicadores de Desempenho

A partir da tabela acima (8 acertos e 8 faltas em 16 acessos):

Taxa de acertos = acertos / total = $8 / 16 = 0,50$ (50%)

Faltas compulsórias = 4 (acessos 1, 3, 9 e 11 — primeira vez que cada linha é usada)

Faltas por conflito = 4 (acessos 6, 8, 13 e 15 — linha já ocupada por outro rótulo)

AMAT = tempo_de_acerto + (taxa_de_faltas $\times$ penalidade_de_falta)

AMAT = $1 + (0,50 \times 8) = 1 + 4 = 5$ ciclos

O resultado de 50% de acerto confirma o que o enunciado antecipa na Seção 5.4: o traço foi montado propositalmente para produzir metade de acertos e metade de faltas.

7. Teste de Conflito — Detector de Números Primos

Com a cache reiniciada (todas as linhas inválidas), percorreu-se v de 0 a 7. Para cada v identificado como primo pelo Componente 17 (detector simplificado por mapa de Karnaugh: $\text{PRIMO} = (\text{NOT } v2$

AND v1) OR (v2 AND v0)), gerou-se um acesso de leitura ao endereço obtido deslocando v três posições à esquerda (v nos bits A5 A4 A3, zeros em A2 A1 A0).

v	v (bin)	Endereço	End. (bin)	Rótulo	Índice	Vál. antes	Tipo de falta
2	010	0x10	010000	010	00	0	compulsória
3	011	0x18	011000	011	00	1	conflito
5	101	0x28	101000	101	00	1	conflito
7	111	0x38	111000	111	00	1	conflito

Os quatro endereços gerados (0x10, 0x18, 0x28, 0x38) têm índice 00 em comum, mas rótulos diferentes. Como a cache usa mapeamento direto, todos disputam exclusivamente a linha 0: o primeiro acesso (v = 2) é uma falta compulsória e ocupa a linha; os três seguintes (v = 3, 5, 7) encontram a linha já válida, porém com rótulo diferente do solicitado, e resultam em falta por conflito — mesmo com as linhas 1, 2 e 3 completamente ociosas durante todo o teste.

Se a cache tivesse 8 linhas em vez de 4, o campo de índice passaria a ter 3 bits ($\log_2(8) = 3$) em vez de 2, deslocando um bit que hoje pertence ao rótulo para o índice. Com isso, os quatro endereços de teste (que diferem justamente nesse bit que migraria de rótulo para índice) passariam a mapear para linhas diferentes entre si, eliminando o conflito observado e convertendo as três faltas por conflito em acertos nas repetições futuras daquele mesmo endereço.

8. Dificuldade Técnica Identificada e Resolvida

Durante os testes iniciais da cache, identificou-se que um único acesso provocava incremento indevido do contador de acertos: a escrita do rótulo/validade de uma linha, disparada por um sinal de nível (não um pulso), alterava combinacionalmente a saída do comparador enquanto o sinal de acesso ainda estava ativo, gerando uma segunda borda de subida no clock do contador dentro da mesma janela de acesso — um caso clássico de clock gating com sinal de nível. A correção adotada foi eliminar qualquer porta lógica no caminho do clock dos elementos síncronos: um único clock limpo passou a alimentar diretamente todos os flip-flops, registradores e o contador, com a condição de atualização (FALTA, ACERTO) migrada para o caminho de dados, por meio de multiplexadores que selecionam entre ‘manter o valor atual’ e ‘carregar o novo valor’.

9. Conclusão

O desenvolvimento da Parte I permitiu aplicar, de forma integrada, conceitos que normalmente são estudados isoladamente em Circuitos Digitais — decodificação de endereços, barramentos compartilhados, lógica combinacional sintetizada a partir de portas básicas e elementos de memória síncronos — montando um subsistema funcional e não apenas componentes avulsos.

O módulo de memória demonstrou, na prática, como um único barramento de endereço, dados e clock pode ser compartilhado por múltiplos dispositivos sem conflito, desde que a lógica de seleção (CS) garanta que apenas um dispositivo responda — e, no caso de escrita, apenas um grave — a cada

acesso. A tabela verdade dos sinais CS confirmou essa exclusividade para as quatro combinações possíveis de A5A4, e o mecanismo de paridade evidenciou, de forma direta, como um erro de integridade de dados pode ser detectado com um circuito relativamente simples, composto por apenas duas árvores XOR de 8 entradas e uma comparação final.

A cache de 4 linhas foi o ponto que mais exigiu domínio dos conceitos de OA3 e OA4: a divisão do endereço em rótulo, índice e deslocamento só faz sentido quando conectada ao comportamento observado no traço de acessos, e os 50% de acerto obtidos confirmaram exatamente o que o enunciado antecipava, validando tanto o circuito quanto o raciocínio por trás da sua construção. O teste de conflito com o detector de primos reforçou esse entendimento ao expor, de forma concreta, a principal limitação do mapeamento direto: quatro acessos geraram quatro faltas por conflito, mesmo com três linhas da cache completamente ociosas, ilustrando por que o número de linhas e a forma de mapeamento afetam diretamente o desempenho de uma cache real.

Um resultado relevante do processo foi a identificação de um problema de sincronismo no contador de acertos, causado por clock gating com sinal de nível — a escrita de uma linha da cache, ainda dentro da mesma janela de acesso, retroalimentava combinacionalmente o comparador e provocava uma contagem indevida. A correção, movendo toda a lógica condicional do caminho de clock para o caminho de dados (através de multiplexadores de autorrealimentação) e mantendo um único clock limpo para todos os elementos síncronos, não foi apenas uma correção pontual, mas uma aplicação direta de um princípio fundamental de projeto síncrono: sinais de habilitação nunca devem gatear diretamente um pino de clock.

De forma geral, a Parte I cumpriu os quatro objetivos de aprendizagem propostos (OA1–OA4), entregando um módulo de memória e uma cache funcionais, testados e documentados, além de evidenciar — através de um problema real encontrado e resolvido — uma compreensão que vai além da simples montagem do circuito, alcançando o entendimento de por que ele se comporta da forma esperada.

Referências

- Enunciado: Laboratório de Circuitos — Projeto Integrador, versão introdutória. Prof. Dr. Herbert Oliveira Rocha, UFRR, 2026.2.
- PATTERSON, D. A.; HENNESSY, J. L. Organização e Projeto de Computadores: a interface hardware/software. (Caminho de dados multiciclo e controle.)
- Documentação do Logisim-Evolution.